Guide de déploiement

La console WeTchah ERP sur un serveur Ubuntu avec Docker et Portainer

Préparer le serveur, installer la console, créer le premier hôtel,
sauvegarder, mettre à jour, changer d'adresse, dépanner.
Chaque étape donne la commande exacte et ce que vous devez voir.

Ce que vous aurez à la fin

Une console qui redémarre seule, en mode production,
un premier hôtel en ligne avec son administrateur,
des sauvegardes nocturnes, et la marche à suivre pour chaque cas courant.
Les commandes ont toutes été exécutées pour de vrai : voir l'annexe C.

Édition du 16 octobre 2026

Sommaire

Sommaire . . . 2

1. Avant de commencer . . . 4
1.1 À quoi sert la console . . . 4
1.2 SQLite ou PostgreSQL ? Les deux, chacun pour son rôle . . . 4
1.3 Quel chemin choisir ? . . . 4
1.4 Comment lire ce guide . . . 5
1.5 Le vocabulaire en clair . . . 5

2. Ce qu'il faut avoir . . . 6
2.1 Le serveur . . . 6
2.2 Ce qui est déjà installé . . . 6
2.3 Les informations à noter avant de commencer . . . 6
2.4 Ouvrir un terminal sur le serveur . . . 7

3. Préparer le serveur . . . 8
3.1 Mettre le serveur à jour . . . 8
3.2 Installer les trois petits outils . . . 8
3.3 Régler l'heure . . . 8
3.4 Vous donner le droit d'utiliser Docker . . . 8
3.5 Ouvrir les ports dans le pare-feu . . . 8
3.6 Créer le réseau Docker « pms » . . . 9
3.7 Créer le dossier des hôtels . . . 9
3.8 Vérifier que le serveur atteint les images des hôtels . . . 9

4. Installer la console (chemin B, production) . . . 10
4.1 Télécharger le code . . . 10
4.2 Créer le fichier des réglages . . . 10
4.3 Fabriquer les clés des notifications (conseillé maintenant) . . . 11
4.4 Créer le raccourci « erp » . . . 11
4.5 Construire et démarrer la console . . . 12
4.6 Préparer la base de la console . . . 12
4.7 Créer votre compte administrateur . . . 12
4.8 Ouvrir la console . . . 13
4.9 Vérifier que la console sait piloter Docker . . . 14

5. Variante : installer la console depuis Portainer . . . 15
5.1 Créer la pile . . . 15
5.2 Vérifier . . . 15

6. Créer le premier établissement . . . 16
6.1 Écran 1 : le propriétaire . . . 17
6.2 Écran 2 : la base de données et le port . . . 18
6.3 Écran 3 : l'établissement et ses modules . . . 19
6.4 La création automatique . . . 19
6.5 Créer le premier compte de l'hôtel . . . 20
6.6 Ce qu'il ne faut pas faire dans Portainer . . . 21

7. Au quotidien . . . 22
7.1 Les commandes utiles . . . 22
7.2 Après une coupure de courant ou un redémarrage du serveur . . . 22
7.3 Que surveiller . . . 22

8. Sauvegarder . . . 23
8.1 Faire une sauvegarde maintenant . . . 23
8.2 Les faire toutes les nuits . . . 23

8.3 Les copier hors du serveur . . . 23
8.4 Restaurer la base de la console . . . 24

9. Mettre à jour . . . 25
9.1 Mettre à jour la console . . . 25
9.2 Mettre à jour un hôtel . . . 25
9.3 Chemin A : mettre à jour la console avec un bouton . . . 25

10. Les cas qui arrivent . . . 28
10.1 J'ai lancé le chemin A (essai) et je veux passer en production sans perdre mes hôtels . . . 28
10.2 L'adresse du serveur a changé (nouvelle IP, nom de domaine) . . . 28
10.3 J'ai oublié le mot de passe de mon compte administrateur . . . 28
10.4 Ajouter d'autres hôtels . . . 29
10.5 Mettre la console en HTTPS avec un nom de domaine . . . 29
10.6 Changer de serveur . . . 29
10.7 Tout désinstaller . . . 29

11. Problèmes courants . . . 30
11.1 Quatre commandes pour comprendre . . . 31

12. Questions fréquentes . . . 32

13. Liste de contrôle . . . 33

Annexe A — Le chemin A : essai rapide . . . 34

Annexe B — Les réglages du fichier . . . 35

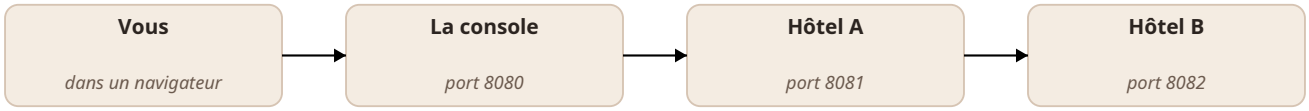
Annexe C — Ce qui a été vérifié . . . 36

1. Avant de commencer

Ce guide explique, pas à pas, comment installer la **console WeTchah ERP** sur un serveur Ubuntu où Docker et Portainer sont déjà installés. Il va de la machine prête jusqu'au **premier hôtel en ligne**, puis il montre comment sauvegarder, mettre à jour et dépanner. Aucune connaissance de Docker n'est nécessaire : chaque commande est donnée toute faite, avec ce que vous devez voir à l'écran quand elle a réussi.

1.1 À quoi sert la console

La console est le « poste de pilotage » de la plateforme. Elle ne gère pas un hôtel : elle **crée et surveille les hôtels**. Quand vous créez un établissement dans la console, elle lance toute seule, sur le même serveur, une application et une base de données rien que pour cet hôtel. Chaque hôtel est donc isolé des autres.



La console crée un hôtel par couple « application + base de données ». Chaque hôtel a son propre port.

1.2 SQLite ou PostgreSQL ? Les deux, chacun pour son rôle

Qui	Base de données utilisée	Pourquoi
La console	**SQLite** — un simple fichier	Elle ne garde que peu de choses : la liste des hôtels, les comptes des administrateurs, les journaux. Aucun serveur de base à installer.
Chaque hôtel	**PostgreSQL** — un conteneur par hôtel	Les réservations, factures, stocks… de l'hôtel. Créé automatiquement avec l'hôtel.

À savoir — Le fichier de la console est `storage/database/database.sqlite`, dans un « volume » Docker (un disque géré par Docker qui survit aux redémarrages). Dans la pile de test (`docker-compose.yml`), un conteneur PostgreSQL nommé `pms-db` démarre aussi, mais la console ne s'en sert pas : vous pouvez l'ignorer.

1.3 Quel chemin choisir ?

	Chemin B — production	Chemin A — essai rapide
Pour qui	Un serveur qui servira vraiment	Essayer, montrer, tester sur une machine
Fichier	`docker-compose.prod.yml`	`docker-compose.yml`
Redémarre seul après une coupure	Oui	Non
Sécurité	Mode production, socket Docker réservé au groupe `docker`	Mode développement, socket ouvert à tous les utilisateurs de la machine
Code	Embarqué dans l'image : rien à compiler à la main	Lu depuis le dossier : il faut créer la base et compiler à la main (annexe A)
Mettre à jour	Reconstruire l'image (9.1)	Bouton « Mettre à jour le code » (9.3)
Où	Chapitres 3 à 9	Annexe A

Important — **Sur un serveur, suivez le chemin B.** Le chemin A sert à essayer. Si vous l'avez déjà lancé, le chapitre 10.1 explique comment passer au chemin B sans perdre vos hôtels.

1.4 Comment lire ce guide

- Un **bloc sombre** contient une commande à copier telle quelle dans le terminal, puis à valider avec la touche Entrée.
- Un **bloc vert « Vous devez voir »** montre ce que le terminal affiche quand tout va bien. S'il affiche autre chose, allez au chapitre 11 (problèmes courants).
- Les mots en `MAJUSCULES` dans une commande, comme `IP_DU_SERVEUR`, sont à remplacer par votre propre valeur.
- Une commande très longue passe à la ligne dans le bloc sombre : c'est **une seule commande**, à copier d'un bloc.
- Chaque étape se termine par une vérification. Ne passez pas à l'étape suivante si la vérification échoue.

1.5 Le vocabulaire en clair

Mot	Ce que cela veut dire
Serveur	L'ordinateur Ubuntu sur lequel on installe tout.
Terminal	La fenêtre où l'on tape des commandes (par SSH depuis votre poste, ou directement sur le serveur).
Docker	Le logiciel qui fait tourner chaque programme dans sa propre « boîte » isolée.
Conteneur	Une de ces boîtes. Exemple : `wetchah_erp-app` est le conteneur de la console.
Image	Le « modèle » à partir duquel Docker fabrique un conteneur. On la construit une fois, on la réutilise.
Volume	Un disque géré par Docker. Il garde les données quand un conteneur est recréé.
Pile (stack)	Un groupe de conteneurs décrits dans un fichier (`docker-compose.prod.yml`) et démarrés ensemble.
Portainer	Une interface web pour voir et piloter Docker avec la souris. Elle sert à surveiller ; l'installation se fait au terminal.
Réseau `pms`	Un réseau interne Docker où la console et tous les hôtels se retrouvent. À créer une fois.
Socket Docker	La « porte de commande » de Docker. La console s'en sert pour créer les hôtels. Qui la possède contrôle le serveur.
Console / établissement	La console = l'outil d'administration. Un établissement = un hôtel avec son application et sa base.
`APP_KEY`	La clé secrète qui chiffre les données de la console. Elle ne doit jamais être perdue ni changée.
Pare-feu	Le filtre qui décide quelles « portes » (ports) du serveur sont ouvertes au réseau.

2. Ce qu'il faut avoir

2.1 Le serveur

Élément	Minimum	Confortable
Système	Ubuntu Server 22.04 ou 24.04, 64 bits	idem
Processeur	2 cœurs	4 cœurs
Mémoire	4 Go	8 Go et plus
Disque	40 Go	100 Go SSD
Internet	Accès sortant vers `ghcr.io` (images des hôtels) et `github.com` (code)	IP fixe ou nom de domaine

Calcul rapide : la console seule prend environ 0,5 Go de mémoire. Chaque hôtel ajoute environ 1 Go de mémoire (application + base) et 1 Go de disque, plus ses données. Pour 5 hôtels, comptez 8 Go de mémoire et 60 Go de disque. Le premier téléchargement des images pèse environ 2 Go.

> **Attention** — **Réservez le serveur à cette plateforme.** La console pilote Docker : elle peut créer, arrêter et supprimer n'importe quel conteneur de la machine. N'y mettez pas d'autres services importants.

2.2 Ce qui est déjà installé

- **Docker** (version 20.10 ou plus) avec le module **Compose v2** (la commande `docker compose`, avec un espace).
- **Portainer**, relié au Docker de ce serveur.
- Vous n'avez **ni PHP, ni Composer, ni Node** à installer : tout est dans l'image de la console.

2.3 Les informations à noter avant de commencer

Remplissez ce tableau : vous en aurez besoin plusieurs fois.

Information	Exemple	Votre valeur
Adresse IP du serveur (ou nom de domaine)	192.168.1.50	
Port de la console	8080	
Plage de ports pour les hôtels	8081 à 8199	
Votre e-mail d'administrateur	admin@exemple.cm	
Adresse du dépôt de code	https://github.com/Adrien-Stage/erp_pms.git	
Dossier du code sur le serveur	/opt/wetchah/wetchah_erp	
Fichier des réglages	/opt/wetchah/erp.env	

> **À savoir** — Pour connaître l'adresse IP du serveur, tapez `hostname -I` : la première adresse affichée est la bonne dans la plupart des cas.

2.4 Ouvrir un terminal sur le serveur

Depuis votre poste, ouvrez un terminal et connectez-vous (remplacez le nom et l'adresse) :

```
ssh NOM_UTILISATEUR@IP_DU_SERVEUR
```

Vous devez arriver sur une invite qui finit par `$`. Toutes les commandes de ce guide se tapent **sur le serveur**, dans cette fenêtre. Pour coller dans un terminal, utilisez `Ctrl + Maj + V` (ou clic droit → Coller).

3. Préparer le serveur

Ces huit étapes se font **une seule fois**, avant d'installer la console.

3.1 Mettre le serveur à jour

```
sudo apt update && sudo apt upgrade -y
```

Cela peut durer quelques minutes. Si le terminal demande un mot de passe, tapez celui de votre compte sur le serveur (rien ne s'affiche quand vous le tapez : c'est normal).

3.2 Installer les trois petits outils

```
sudo apt install -y git openssl curl
```

Si ces outils sont déjà installés, la commande le dit et ne fait rien.

3.3 Régler l'heure

```
sudo timedatectl set-timezone Africa/Douala
```

Les journaux et les sauvegardes planifiées se règlent sur l'horloge du serveur. Vérifiez avec `date`.

3.4 Vous donner le droit d'utiliser Docker

Sans ce droit, chaque commande Docker devrait commencer par `sudo`. Ajoutez-vous au groupe `docker` :

```
sudo usermod -aG docker $USER
```

Fermez votre session puis reconnectez-vous (tapez `exit`, puis reconnectez-vous en SSH) pour que ce changement soit pris en compte. Vérifiez :

```
docker ps
```

> **Vous devez voir**
> `CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES`
> `(la liste de vos conteneurs, dont Portainer ; elle peut être courte)`

3.5 Ouvrir les ports dans le pare-feu

Regardez d'abord si le pare-feu `ufw` est actif :

```
sudo ufw status
```

S'il répond `Status: inactive`, passez à l'étape 3.6. S'il répond `Status: active`, ouvrez les trois plages suivantes :

```
sudo ufw allow 8080/tcp
sudo ufw allow 8081:8199/tcp
sudo ufw allow 9081:9199/tcp
```

Port	Usage
`8080`	La console
`8081 à 8199`	Les applications des hôtels — un port par hôtel
`9081 à 9199`	Les sites vitrines des hôtels (module « site web ») — port de l'hôtel + 1000

Attention — **Docker contourne ufw.** Un port publié par Docker reste joignable même si ufw ne l'ouvre pas : Docker écrit ses propres règles. Pour vraiment limiter l'accès (par exemple à votre réseau local), faites-le au niveau du routeur ou du pare-feu de votre hébergeur.

3.6 Créer le réseau Docker « pms »

C'est le réseau où la console et tous les hôtels se retrouvent. On le crée une fois :

```
docker network create pms
```

Vous devez voir

```
3f9c0e7c1d5a…   (une longue suite de lettres et de chiffres)
```

Si Docker répond `network with name pms already exists`, c'est que le réseau existe déjà : tout va bien.

3.7 Créer le dossier des hôtels

La console écrit dans ce dossier un petit fichier de configuration par hôtel. Il contient les **clés secrètes** de chaque hôtel : gardez-le privé et sauvegardez-le (chapitre 8).

```
sudo mkdir -p /var/wetchah/tenants
sudo chown 33:33 /var/wetchah/tenants
```

Ce dossier est celui **par défaut**. À la création d'un hôtel, on peut en choisir un autre pour cet hôtel seulement (6.2) ; il doit alors être monté dans la console, comme celui-ci.

Important — Ne sautez pas la deuxième commande. La console travaille avec l'utilisateur `www-data` (numéro 33). Sans elle, la création du premier hôtel échoue avec « Impossible de créer le répertoire ».

3.8 Vérifier que le serveur atteint les images des hôtels

```
docker pull ghcr.io/adrien-stage/villa_b:latest
```

Vous devez voir

```
latest: Pulling from adrien-stage/villa_b
…
Status: Downloaded newer image for ghcr.io/adrien-stage/villa_b:latest
```

Le premier téléchargement prend quelques minutes. S'il échoue ou reste bloqué, réglez d'abord votre connexion (DNS, proxy, pare-feu sortant) : sans cela, aucun hôtel ne pourra être créé.

4. Installer la console (chemin B, production)

Ce chapitre installe la console avec le terminal. Si vous préférez cliquer dans Portainer, faites les étapes 4.1 à 4.4, puis lisez le chapitre 5 à la place de 4.5.

4.1 Télécharger le code

```
sudo mkdir -p /opt/wetchah && sudo chown $USER: /opt/wetchah
git clone https://github.com/Adrien-Stage/erp_pms.git /opt/wetchah/wetchah_erp
cd /opt/wetchah/wetchah_erp && git checkout main
```

Le téléchargement est gros (environ 140 Mo) parce que le dépôt embarque le client Docker que la console utilise : c'est voulu.

> **À savoir** — Si le dépôt est **privé**, GitHub demande un nom d'utilisateur et un mot de passe. Le mot de passe est alors un **jeton d'accès personnel** (GitHub → Settings → Developer settings → Personal access tokens, droit « Contents : lecture seule »).

4.2 Créer le fichier des réglages

Tous les réglages de la console vivent dans **un seul fichier**, rangé hors du dépôt pour que vos secrets n'y soient jamais enregistrés.

```
cp /opt/wetchah/wetchah_erp/.env.production.example /opt/wetchah/erp.env
chmod 600 /opt/wetchah/erp.env
```

Fabriquez maintenant la clé secrète de la console :

```
echo "base64:$(openssl rand -base64 32)"
```

> **Vous devez voir**
> `base64:Qk3x…(43 caractères suivis d'un signe =)`

Copiez cette ligne entière (elle commence par `base64:`) dans votre gestionnaire de mots de passe. C'est la valeur de `APP_KEY`.

> **Important** — **Gardez la clé `APP_KEY` hors du serveur, et ne la changez jamais.** Sans elle, la base de la console n'est plus lisible, même avec toutes vos sauvegardes.

Ouvrez le fichier avec l'éditeur simple `nano` :

```
nano /opt/wetchah/erp.env
```

Modifiez **seulement ces lignes** (déplacez-vous avec les flèches du clavier) :

Ligne	Quoi mettre	Exemple
`APP_KEY=`	La clé fabriquée ci-dessus, collée juste après le signe =	APP_KEY=base64:Qk3x…=
`APP_URL=`	http:// + IP du serveur + :8080	APP_URL=http://192.168.1.50:8080
`TENANTS_PUBLIC_HOST=`	L'IP (ou le nom de domaine) du serveur, **sans** http:// ni port	TENANTS_PUBLIC_HOST=192.168.1.50
`VAPID_… (3 lignes)`	Clés des notifications — voir 4.3. Vous pouvez laisser vide pour un essai.	
`MAIL_…`	Laissez `log` pour démarrer. Renseignez Resend plus tard pour envoyer de vrais courriels.	MAIL_MAILER=log

Pour enregistrer : `Ctrl + O`, puis `Entrée`. Pour quitter : `Ctrl + X`. Laissez toutes les autres lignes telles quelles.

> **Important — TENANTS_PUBLIC_HOST est le réglage qui distingue un serveur d'un poste.** S'il reste à `localhost`, les boutons « Ouvrir » de la console mènent à votre propre poste, et le site vitrine d'un hôtel refuse ses formulaires.

4.3 Fabriquer les clés des notifications (conseillé maintenant)

Ces deux clés servent aux notifications des hôtels. Elles sont **inscrites dans chaque hôtel à sa création** : les changer plus tard oblige à recréer les hôtels. Fabriquez-les donc dès maintenant :

```
docker run --rm node:22-slim npx --yes web-push generate-vapid-keys
```

Vous devez voir

```
Public Key:
BFd16f19A1ab-m_GDX__Xd9Oc_Fk0R5dld_oK4JW…

Private Key:
UzchiDiiu-X6iH3WTLECjoydmz2YnFBoMH9W…
```

Collez la **Public Key** après `VAPID_PUBLIC_KEY=`, la **Private Key** après `VAPID_PRIVATE_KEY=`, et mettez un e-mail de contact après `VAPID_SUBJECT=mailto:`. Gardez ces clés dans votre gestionnaire de mots de passe. Rouvrez `nano` pour les coller si le fichier est déjà fermé.

4.4 Créer le raccourci « erp »

Toutes les commandes suivantes commencent par la même longue formule. Créons un raccourci nommé `erp` qui la contient :

```
echo "alias erp='docker compose -f /opt/wetchah/wetchah_erp/docker-compose.prod.yml --env-file /opt/wetchah/erp.env'" >> ~/.bashrc
source ~/.bashrc
```

Vérifiez que le fichier de réglages est complet :

```
erp config -q && echo "Réglages OK"
```

Vous devez voir

```
Réglages OK
```

Si le terminal répond `required variable APP_KEY is missing a value`, la ligne `APP_KEY=` est restée vide : retournez à l'étape 4.2.

> **À savoir** — Après cela, `erp ps` veut dire « montre l'état de la console », `erp logs` « montre le journal », etc. Ce raccourci reste disponible à chaque connexion.

4.5 Construire et démarrer la console

```
erp up -d --build
```

La première fois, comptez 5 à 15 minutes : Docker fabrique l'image (système, PHP, bibliothèques, compilation des écrans). Le terminal défile longuement : laissez-le travailler.

Vous devez voir

```
 Volume wetchah-erp_erp_storage Created
 Container wetchah_erp-app Creating
 Container wetchah_erp-app Started
```

Attendez environ une minute, puis vérifiez l'état :

```
erp ps
```

Vous devez voir

```
NAME             IMAGE                COMMAND   SERVICE   CREATED          STATUS
  PORTS
wetchah_erp-app   wetchah-erp:latest   …         app       1 minute ago     Up 1 minute
(healthy)     0.0.0.0:8080->80/tcp
```

> **À savoir** — « `healthy` » veut dire que Docker a vérifié que la page de connexion répond. Pendant la première minute, vous verrez « `health: starting` » : patientez.

4.6 Préparer la base de la console

La console démarre avec une base vide. Cette commande y crée les tables :

```
erp exec -u www-data app php artisan migrate --force
```

Vous devez voir

```
INFO  Running migrations.

…
2026_10_01_200000_create_permission_matrix_versions_table ..... DONE
2026_10_02_100000_create_tenant_interventions_table ........... DONE
```

> **Important** — **Ne lancez jamais « db:seed » en production.** Ce programme crée des comptes d'essai `admin` / `admin` et `owner` / `owner`, et le premier donne la main sur Docker, donc sur le serveur. Créez plutôt votre vrai compte à l'étape suivante.

4.7 Créer votre compte administrateur

```
erp exec -u www-data app php artisan erp:creer-admin
```

La commande pose quatre questions. Répondez à chacune puis validez avec Entrée :

Question	Votre réponse
Nom et prénom	Marie Abena
Adresse e-mail (sert à se connecter)	marie@exemple.cm
Mot de passe (12 caractères minimum)	Un mot de passe long et unique — il ne s'affiche pas quand vous le tapez
Retapez le mot de passe	Le même, une seconde fois

> **Vous devez voir**
> `Compte créé : marie@exemple.cm. Vous pouvez vous connecter.`

Si elle répond « Cette adresse est déjà utilisée », ce compte existe déjà (voir 10.3 si le mot de passe est perdu). Le mot de passe saisi n'est enregistré nulle part en clair.

4.8 Ouvrir la console

Sur votre poste, ouvrez un navigateur et tapez l'adresse du serveur, avec le port 8080 :

```
http://IP_DU_SERVEUR:8080
```

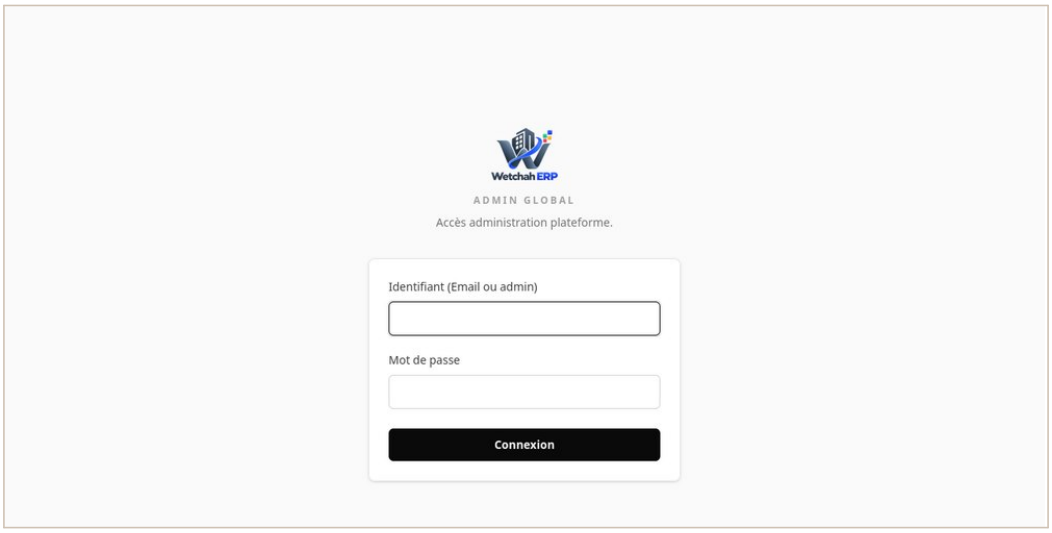


Figure 1 — La page de connexion de la console. Le champ s'appelle « Identifiant (Email ou admin) » : tapez votre adresse e-mail.

Connectez-vous avec l'adresse e-mail et le mot de passe de l'étape 4.7. Vous arrivez sur la **Supervision** :

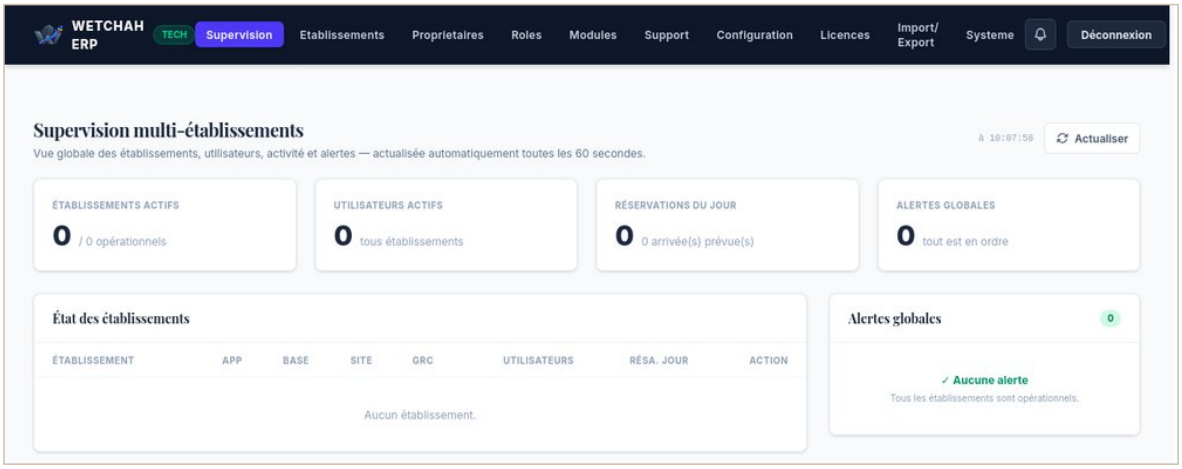


Figure 2 — La Supervision d'une console neuve : aucun établissement, aucune alerte. C'est exactement ce qu'on doit voir.

4.9 Vérifier que la console sait piloter Docker

Sans cela, elle ne pourra créer aucun hôtel. Deux contrôles :

```
erp exec -u www-data app docker ps
```

Vous devez voir

```
CONTAINER ID   IMAGE                …   NAMES
…              wetchah-erp:latest   …   wetchah_erp-app
```

```
ls -l /var/run/docker.sock
```

Vous devez voir

```
srw-rw---- 1 root docker 0 … /var/run/docker.sock
```

Attention — Si le second contrôle affiche `srw-rw-rw-` (trois fois `rw`), le socket est ouvert à tout utilisateur de la machine. Vérifiez que la ligne `ERP_OUVRIR_SOCKET_DOCKER: "0"` est bien présente dans `docker-compose.prod.yml`, puis tapez `erp up -d --force-recreate`.

5. Variante : installer la console depuis Portainer

Au lieu de l'étape 4.5, vous pouvez demander à Portainer de construire et de démarrer la console. Le résultat est le même ; seul le moyen change.

> **Attention** — **Choisissez une méthode et gardez-la.** Si vous démarrez la console depuis Portainer, ne tapez plus `erp up` : les deux ne se mélangent pas. Les écrans de Portainer n'ont pas pu être rejoués pour ce guide : les libellés ci-dessous peuvent légèrement changer d'une version à l'autre.

5.1 Créer la pile

1. Ouvrez Portainer dans votre navigateur et choisissez l'environnement **local**.
2. Dans le menu de gauche, cliquez sur **Stacks**, puis sur **+ Add stack**.
3. **Name** : tapez exactement `wetchah-erp` (ce nom devient le préfixe du volume de la console).
4. **Build method** : choisissez **Repository**.
5. **Repository URL** : `https://github.com/Adrien-Stage/erp_pms`.
6. **Repository reference** : `refs/heads/main`.
7. **Compose path** : `docker-compose.prod.yml`.
8. Dépôt privé : activez **Authentication** et saisissez votre nom d'utilisateur GitHub et votre jeton d'accès.
9. **Environment variables** : cliquez sur **Advanced mode**, puis collez le contenu de votre fichier de réglages (affichez-le avec `cat /opt/wetchah/erp.env`). Les lignes vides et les commentaires ne gênent pas.
10. Laissez **GitOps updates** désactivé : une mise à jour automatique redémarrerait la console sans prévenir.
11. Cliquez sur **Deploy the stack**.

Portainer fabrique l'image : le journal reste silencieux 5 à 15 minutes. Patientez, sans fermer la page.

5.2 Vérifier

Dans **Containers**, la ligne `wetchah_erp-app` doit passer à **running**, puis à **healthy** au bout d'une minute. Il reste ensuite à préparer la base et à créer votre compte. Sans le raccourci `erp`, tapez les commandes complètes :

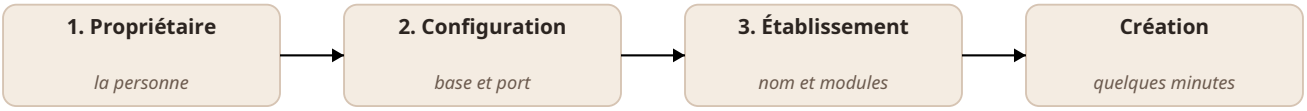
```
docker exec -u www-data wetchah_erp-app php artisan migrate --force
docker exec -it -u www-data wetchah_erp-app php artisan erp:creer-admin
```

Retrouvez ensuite les étapes 4.8 et 4.9, qui sont identiques. Pour les chapitres suivants, remplacez `erp exec -u www-data app` par `docker exec -u www-data wetchah_erp-app`, et `erp up -d --build` par **Pull and redeploy** dans la pile Portainer.

> **À savoir** — Le fichier de réglages n'est pas lu par Portainer : c'est le contenu collé dans **Environment variables** qui compte. Pour modifier un réglage plus tard, ouvrez la pile → **Editor** → modifiez la variable → **Update the stack**.

6. Créer le premier établissement

Connectez-vous à la console, puis ouvrez le menu **Etablissements** et cliquez sur **Nouvel Établissement**. L'assistant a trois écrans.



Les trois écrans de l'assistant, puis la création automatique de l'hôtel.

6.1 Écran 1 : le propriétaire

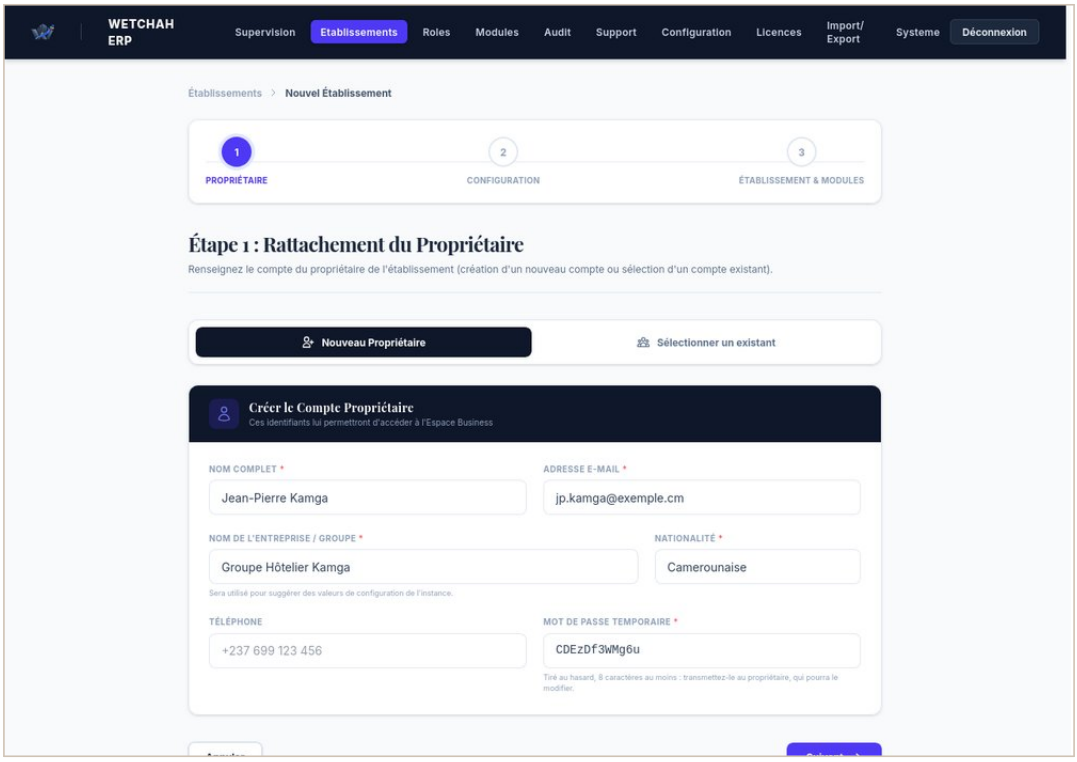


Figure 3 — Écran 1 : le compte du propriétaire de l'hôtel. Le mot de passe temporaire est proposé : transmettez-le au propriétaire.

Champ	Que mettre
Nom complet, Adresse e-mail	Ceux du propriétaire de l'hôtel.
Nom de l'entreprise / Groupe	Le groupe ou la société. Il sert à proposer un nom de base de données.
Nationalité	Obligatoire.
Mot de passe temporaire	Proposé au hasard, 8 caractères minimum. À transmettre au propriétaire.

Si le propriétaire possède déjà un autre hôtel, cliquez sur **Sélectionner un existant** au lieu de créer un nouveau compte.

6.2 Écran 2 : la base de données et le port

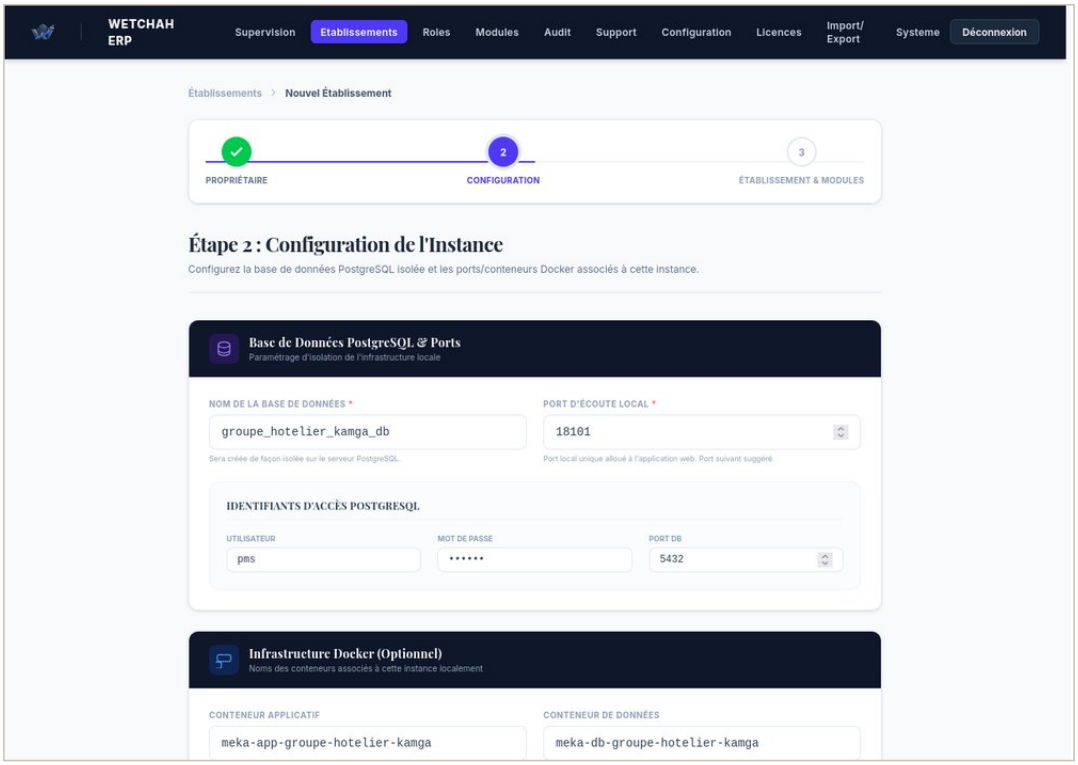


Figure 4 — Écran 2 : tout est proposé. Vérifiez le port et remplacez le mot de passe de la base.

Champ	Que mettre
Nom de la base de données	Proposé à partir du nom du groupe. Il doit être unique.
Port d'écoute local	**Un port libre dans la plage ouverte au pare-feu** (8081 à 8199). C'est l'adresse de l'hôtel : `http://IP_DU_SERVEUR:PORT`. La console vérifie qu'il est libre.
Identifiants PostgreSQL — Mot de passe	**Remplacez la valeur proposée par un mot de passe long.** Tous les hôtels partagent le réseau `pms` : un mot de passe faible laisserait un hôtel atteindre la base d'un autre.
Dossier des établissements	**Laissez vide** pour utiliser `/var/wetchah/tenants` (3.7). Pour en choisir un autre, cliquez sur **Parcourir…** : une fenêtre liste les dossiers **du serveur** (pas de votre poste) ; descendez, remontez, créez un sous-dossier, puis **Choisir ce dossier**. La console prévient si elle ne peut pas y écrire, ou si le dossier n'est pas monté depuis le serveur (il disparaîtrait avec le conteneur de la console).
Utilisateur, Port DB	Laissez les valeurs proposées.
Conteneur applicatif / de données	Laissez les noms proposés.

6.3 Écran 3 : l'établissement et ses modules

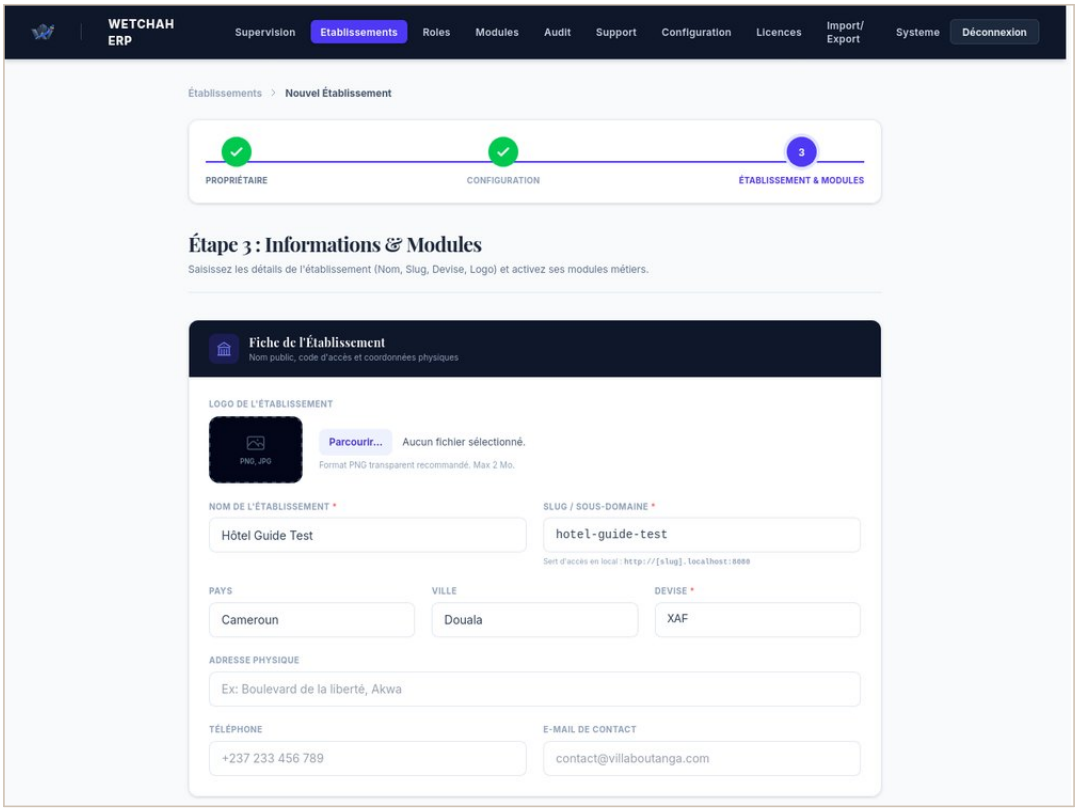


Figure 5 — Écran 3 : nom, adresse, couleurs de l'hôtel, puis (plus bas) les modules à installer.

Champ	Que mettre
Nom de l'établissement	Le nom public. Le « slug » se remplit tout seul.
Slug	Court, en minuscules, sans espace. **Il ne se change plus** : tous les noms Docker de l'hôtel en dépendent.
Pays, Ville, Devise	Cameroun, Douala, XAF par défaut.
Modules	Cochez ce que l'hôtel utilise. Cocher le « site web » active aussi l'« API ». Chaque module de plus rallonge un peu la création.

Descendez en bas de la page et cliquez sur **Créer l'Établissement**.

6.4 La création automatique

La page de l'hôtel s'ouvre et affiche, en direct, ce que fait la console : elle télécharge l'image de l'hôtel, écrit sa configuration, démarre l'application et la base, puis attend qu'elles répondent.

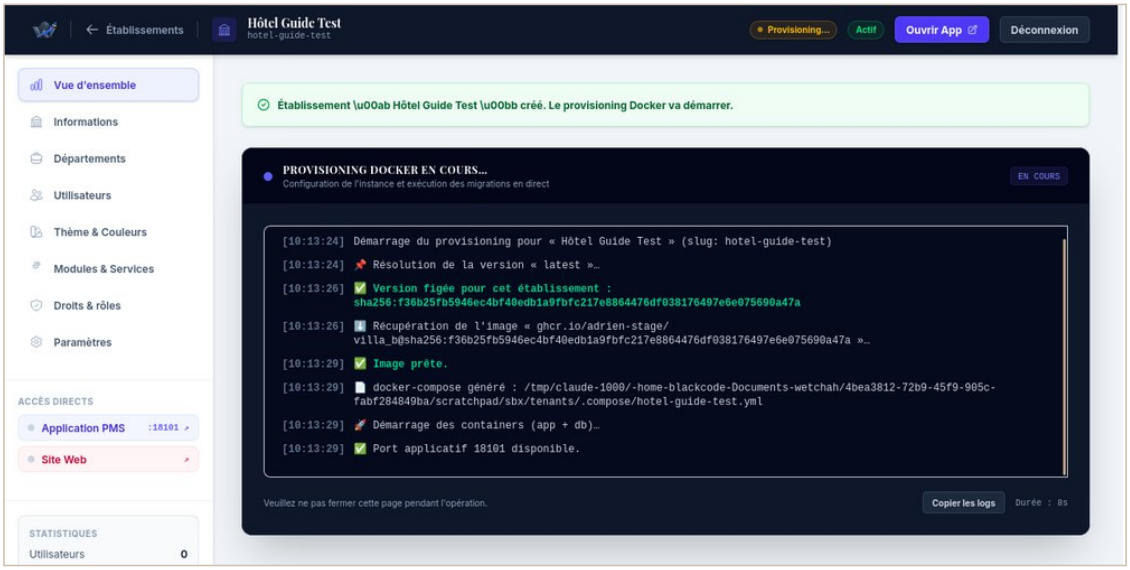


Figure 6 — Pendant la création : le journal en direct. Comptez 2 à 5 minutes (davantage au premier hôtel, à cause du téléchargement).

> **Attention** — **Ne fermez pas la page pendant la création.** Si vous l'avez fermée sans le vouloir, rouvrez la fiche de l'hôtel (menu Etablissements → l'hôtel) : la création reprend et se termine toute seule. Si la page indique une erreur, un bouton rouge **Relancer le provisioning** apparaît.

Quand c'est fini, l'état passe à **Conteneurs actifs** et la carte « Application PMS » affiche **En ligne**.

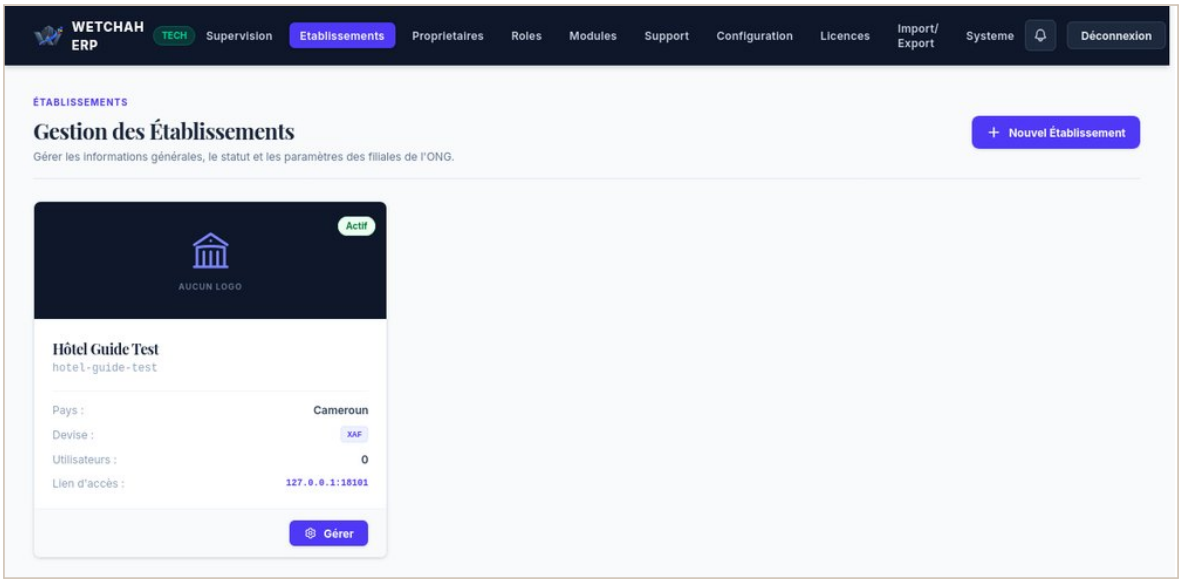


Figure 7 — La liste des établissements. Le « Lien d'accès » doit afficher l'adresse de votre serveur, pas localhost : c'est TENANTS_PUBLIC_HOST qui le règle.

6.5 Créer le premier compte de l'hôtel

L'hôtel vient d'être créé : **il n'a encore aucun utilisateur**. Le compte propriétaire sert à la console, pas à l'application de l'hôtel. Le premier compte de l'hôtel — celui de son administrateur informatique — se crée depuis la console :

1. Ouvrez la fiche de l'hôtel, puis le menu **Droits & rôles**.
2. Cliquez sur l'onglet **Comptes administrateurs**.
3. Remplissez **Nom complet** et **Adresse e-mail**. Laissez **Mot de passe** vide : la console en tire un au hasard.
4. Cliquez sur **Créer l'administrateur**.

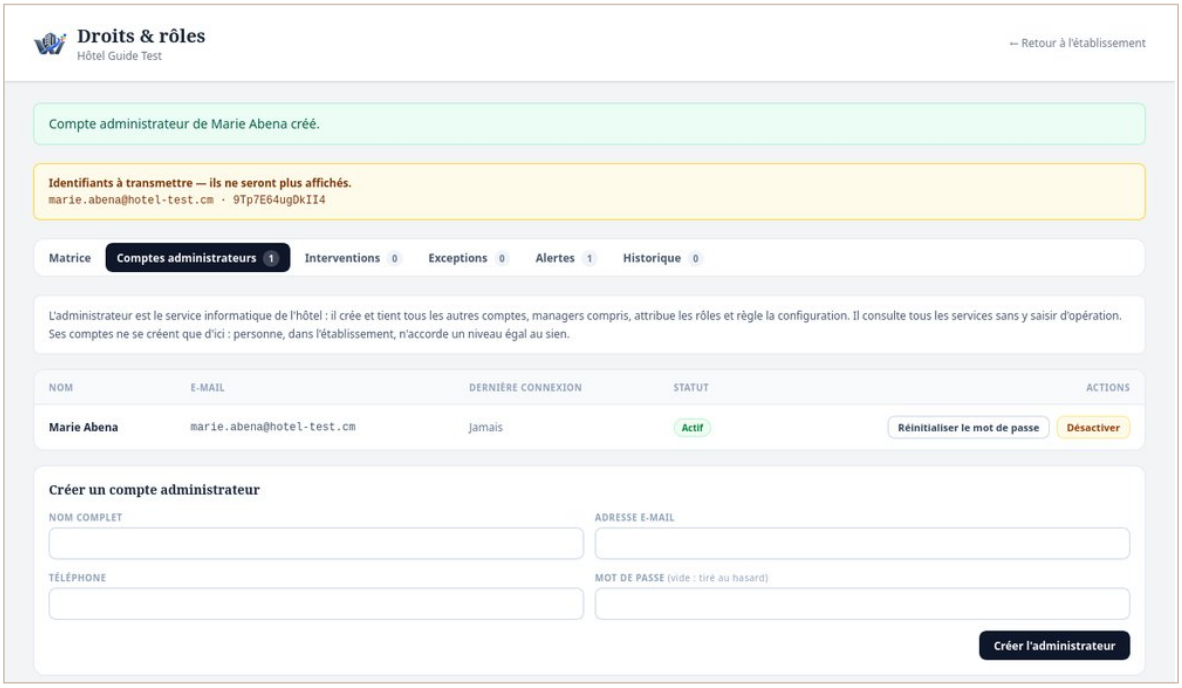


Figure 8 — Le compte est créé. L'encadré jaune montre l'e-mail et le mot de passe : ils ne seront plus affichés. Notez-les et transmettez-les.

L'administrateur se connecte ensuite sur `http://IP_DU_SERVEUR:PORT` (le port de l'hôtel), et crée lui-même tous les autres comptes — managers, réceptionnistes… — dans l'application.

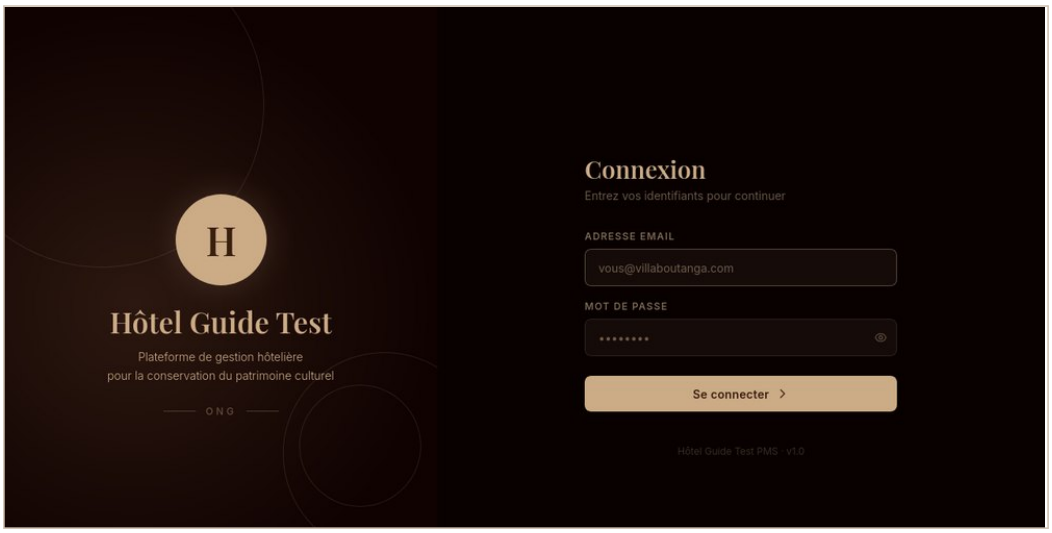


Figure 9 — La page de connexion de l'hôtel, aux couleurs choisies à l'écran 3.

> **À savoir** — Dans Portainer, vous verrez les conteneurs `meka-erp-NOM-app` et `meka-erp-NOM-db` **hors de toute pile** (mention « Unmanaged » ou « Limited »). C'est normal : c'est la console qui les a créés.

6.6 Ce qu'il ne faut pas faire dans Portainer

- **Ne supprimez, ne modifiez ni ne mettez à jour un hôtel depuis Portainer.** La console tient le registre de chaque hôtel (port, version, secrets) ; une action faite à côté la désynchronise. Utilisez la console.
- **Ne supprimez jamais** les volumes `meka-erp-NOM_meka_erp_NOM_pgdata` et `meka-erp-NOM_meka_erp_NOM_storage` : ils contiennent les données de l'hôtel.
- Utilisez Portainer pour **regarder** : état, journaux, consommation de mémoire.

7. Au quotidien

7.1 Les commandes utiles

À taper depuis n'importe quel dossier, grâce au raccourci `erp` (voir 4.4).

Je veux…	Commande
Voir si la console tourne	`erp ps`
Lire son journal, en direct (Ctrl + C pour sortir)	`erp logs -f app`
Lire les erreurs de l'application	`erp exec app tail -n 50 storage/logs/laravel.log`
Arrêter la console (les hôtels continuent)	`erp stop`
La redémarrer	`erp start`
La recréer après une modification de réglage	`erp up -d`
Voir les hôtels qui tournent	`docker ps --filter name=meka-erp`
Voir la place prise par Docker	`docker system df`

7.2 Après une coupure de courant ou un redémarrage du serveur

Rien à faire : la console redémarre seule (réglage `restart: unless-stopped`). Les hôtels aussi, car Docker relance les conteneurs qui tournaient. Vérifiez avec `erp ps` et `docker ps`. Si Docker lui-même ne démarre pas au démarrage du serveur : `sudo systemctl enable docker`.

7.3 Que surveiller

Quoi	Comment	Fréquence
La console répond	Ouvrir `http://IP_DU_SERVEUR:8080`	Chaque jour
Les hôtels sont en ligne	Menu Supervision de la console : une ligne par hôtel	Chaque jour
Place disque	`df -h /` — alerte au-dessus de 80 %	Chaque semaine
Sauvegardes	Le fichier du jour existe dans `/srv/sauvegardes` et a été copié ailleurs	Chaque semaine

8. Sauvegarder

Une sauvegarde utile est une copie **hors du serveur**. Si le disque meurt, tout ce qui est sur le serveur meurt avec lui. Il y a quatre choses à protéger :

Élément	Où il est	Pourquoi
Base de la console	Volume `wetchah-erp_erp_storage`	La liste des hôtels, leurs mots de passe de base, les comptes administrateurs.
Fichiers des hôtels	`/var/wetchah/tenants`	Les clés secrètes de chaque hôtel. **Perdues, elles rendent des données chiffrées irrécupérables.**
Données des hôtels	Menu **Sauvegardes** de la console	La console en fait des copies de la base de chaque hôtel, rangées dans le volume de la console (donc incluses dans la copie ci-dessous).
Clé APP_KEY, clés VAPID	Votre gestionnaire de mots de passe	Sans APP_KEY, la base de la console n'est plus lisible.

8.1 Faire une sauvegarde maintenant

Un petit programme fourni avec le code fait les trois copies d'un coup, sans arrêter la console :

```
sudo /opt/wetchah/wetchah_erp/scripts/sauvegarder-erp.sh
```

Vous devez voir

```
[2026-10-16 02:30:00] Sauvegarde de wetchah_erp-app vers /srv/sauvegardes
  base de la console : console-2026-10-16_0230.sqlite
  volume storage     : console-storage-2026-10-16_0230.tgz
  dossier des hôtels : tenants-2026-10-16_0230.tgz
[2026-10-16 02:30:04] Terminé.
```

Les fichiers sont dans `/srv/sauvegardes`. Le programme garde les 14 dernières copies de chaque sorte et efface les plus anciennes.

8.2 Les faire toutes les nuits

```
sudo crontab -e
```

Le terminal propose un éditeur (choisissez nano si on vous le demande). Ajoutez cette ligne **tout en bas** du fichier : elle lance la sauvegarde chaque nuit à 2 h 30.

```
30 2 * * * /opt/wetchah/wetchah_erp/scripts/sauvegarder-erp.sh >> /var/log/sauvegarde-erp.log
2>&1
```

Enregistrez (`Ctrl + O`, Entrée) et quittez (`Ctrl + X`). Le lendemain, vérifiez avec `ls -l /srv/sauvegardes`.

8.3 Les copier hors du serveur

Choisissez **une** de ces solutions, et testez-la :

Solution	Exemple
Vers un autre ordinateur, par le réseau	`rsync -av /srv/sauvegardes/ utilisateur@AUTRE_MACHINE:/sauvegardes-erp/`
Vers un disque USB branché sur le serveur	`rsync -av /srv/sauvegardes/ /media/USB/sauvegardes-erp/`
Vers un stockage en ligne	L'outil `rclone` (documentation sur rclone.org)

> **Attention** — Une sauvegarde qui reste sur le serveur n'en est pas une. Faites aussi, **une fois**, un essai de restauration sur une autre machine : c'est le seul moyen d'être sûr qu'elle fonctionne.

8.4 Restaurer la base de la console

À faire si la base de la console est abîmée ou perdue. Remplacez la date par celle du fichier choisi dans `/srv/sauvegardes`.

1. Arrêtez la console : `erp stop`.
2. Remettez le fichier à sa place :

```
docker run --rm -v wetchah-erp_erp_storage:/donnees -v /srv/sauvegardes:/sauv:ro alpine sh -c 'cp /sauv/console-AAAA-MM-JJ_HHMM.sqlite /donnees/database/database.sqlite && rm -f /donnees/database/database.sqlite-wal /donnees/database/database.sqlite-shm && chown 33:33 /donnees/database/database.sqlite'
```

1. Redémarrez : `erp start`.
2. Ouvrez la console et vérifiez que vos hôtels sont bien listés.

Si le volume est entièrement perdu, restaurez d'abord l'archive `console-storage-….tgz` dans le volume, puis le dossier `/var/wetchah/tenants` (`sudo tar xzf tenants-….tgz -C /var/wetchah`, puis `sudo chown -R 33:33 /var/wetchah/tenants`).

9. Mettre à jour

9.1 Mettre à jour la console

> **Important** — **Faites d'abord une sauvegarde** (8.1) : une mise à jour peut modifier la base de la console.

On donne un **numéro de version** à chaque mise à jour. Ainsi l'ancienne version reste disponible si la nouvelle pose problème. Ici, le numéro est la date du jour.

1. Téléchargez le nouveau code : `cd /opt/wetchah/wetchah_erp && git pull`.
2. Fabriquez la nouvelle version sous son numéro : `ERP_VERSION=$(date +%F) erp build`.
3. Ouvrez le fichier de réglages (`nano /opt/wetchah/erp.env`) et **ajoutez tout en bas** la ligne `ERP_VERSION=2026-10-16` (la date du jour). Enregistrez.
4. Démarrez la nouvelle version : `erp up -d`. La console est indisponible une à deux minutes ; **les hôtels ne sont pas touchés**.
5. Appliquez les changements de base : `erp exec -u www-data app php artisan migrate --force`.

> **Vous devez voir**
> `INFO  Nothing to migrate.   (ou la liste des nouvelles migrations, terminées par DONE)`

Vérifiez : `erp ps` doit afficher l'image `wetchah-erp:2026-10-16` (le numéro choisi), à l'état `healthy`.

Revenir en arrière

Remettez dans le fichier de réglages le numéro de l'ancienne version (ou effacez la ligne `ERP_VERSION` pour revenir à `latest`), puis tapez `erp up -d`. L'ancienne image est encore là : le retour prend moins d'une minute. Si des migrations avaient été appliquées, restaurez aussi la base (8.4), car une migration n'est pas toujours réversible.

> **À savoir** — Le code n'est relu que quand le conteneur démarre. Ne modifiez jamais un fichier *dans* un conteneur qui tourne : construisez une nouvelle image.

9.2 Mettre à jour un hôtel

Les hôtels ont leur propre numéro de version, indépendant de celui de la console. Pour mettre un hôtel à jour :

1. Dans la console, ouvrez la fiche de l'hôtel, menu **Paramètres**.
2. Dans la zone **Version de l'application**, cliquez sur **Vérifier les mises à jour**.
3. Choisissez la version, puis **Lancer la mise à jour**. Le journal défile ; ne fermez pas la page.

> **Vous devez voir**
> ```
> Container mis à jour.
> Tentative 2/30 — container running, HTTP: 502…
> Application opérationnelle (HTTP 302).
> Établissement « … » mis à jour avec succès.
> ```

L'hôtel est indisponible une minute environ. Les mêmes écrans existent pour le site vitrine et le module de contrôle de gestion, quand ils sont activés.

9.3 Chemin A : mettre à jour la console avec un bouton

Dans la pile d'essai, le code est un dossier cloné depuis git. Un bouton de la console fait à votre place le « git pull » sur **la branche où le code se trouve** (par exemple `main` ou `develop`), puis les migrations.

1. Connectez-vous à la console, ouvrez le menu **Systeme** (en haut à droite).
2. La carte indique la **branche**, la **version installée** et l'**adresse du dépôt**.
3. Cliquez sur **Vérifier les nouveautés** : la liste des commits en attente s'affiche, ou « Le code est à jour ».
4. Cliquez sur **Mettre à jour le code (git pull)**. Le journal défile en direct ; ne fermez pas la page avant « Terminé ».

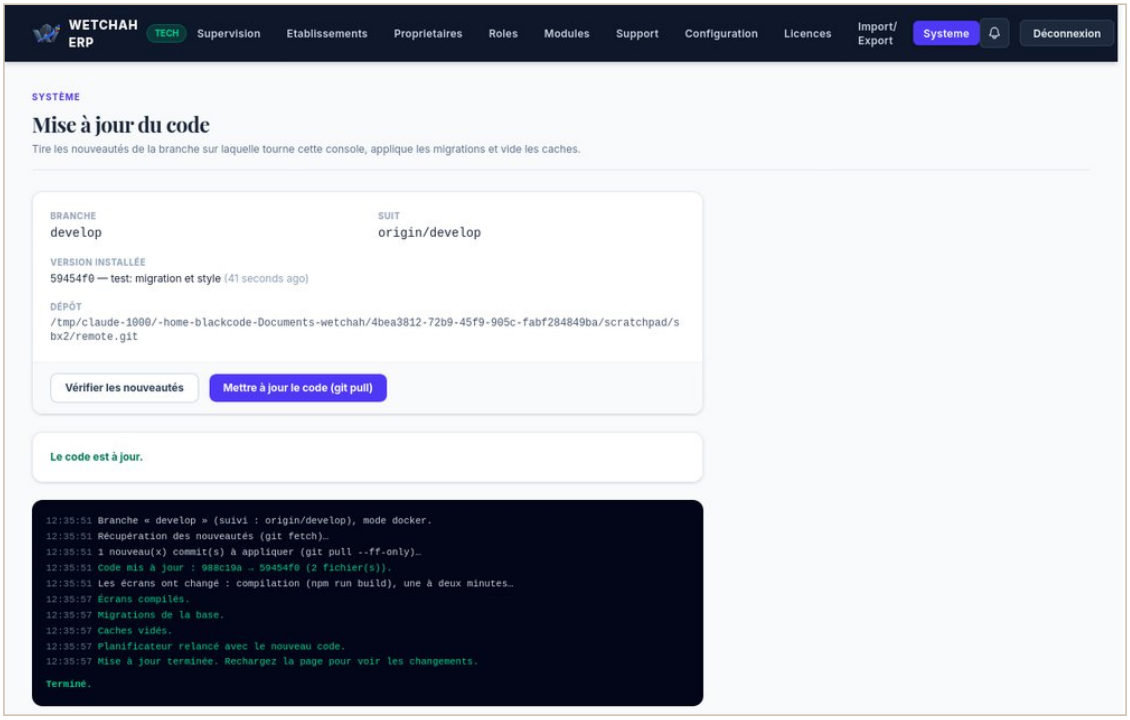


Figure 10 — L'onglet Système après une mise à jour : branche, version, résultat de la vérification et journal.

Ce que fait le bouton, dans l'ordre : récupère les nouveautés (`git fetch`) ; tire en « avance rapide » (`git pull --ff-only`, sans jamais fusionner ni écraser) ; recompile les écrans si des fichiers d'écran ont changé ; applique les migrations (`migrate --force`) ; vide les caches ; relance le planificateur. Aucune commande à taper.

Le bouton refuse quand…	Que faire
Des fichiers du code ont été modifiés à la main	La liste est affichée. Annulez ces changements (`git checkout -- fichier`) ou enregistrez-les (`git commit`), puis recommencez.
La branche a divergé du dépôt (commits locaux absents du dépôt)	Une mise à jour sans conflit est impossible : à résoudre à la main avec git.
Aucune branche n'est active (HEAD détaché), ou elle ne suit aucune branche distante	`git checkout BRANCHE`, ou `git branch --set-upstream-to=origin/BRANCHE`.
Le dépôt est en SSH	La console n'a pas de clés SSH : `git remote set-url origin https://github.com/ORGANISATION/DEPOT.git`.
Le dépôt est privé	Mettez un jeton d'accès dans l'adresse : `git remote set-url origin https://JETON@github.com/ORGANISATION/DEPOT.git`. Le jeton n'est jamais affiché à l'écran.

> **Attention — Le bouton ne met pas à jour les dépendances.** Si `composer.lock` a changé, le journal avertit : reconstruisez alors l'image (`docker compose build app`, puis `docker compose up -d`).

Important — Sur une console de production (chemin B), le code est dans l'image, pas dans un dépôt git. L'onglet Système l'explique et ne propose pas le bouton : on met à jour comme au 9.1, en reconstruisant l'image. Les mises à jour qui changent la base (migrations) se font alors par `erp exec -u www-data app php artisan migrate --force`.

10. Les cas qui arrivent

10.1 J'ai lancé le chemin A (essai) et je veux passer en production sans perdre mes hôtels

Les hôtels déjà créés vivent dans Docker et dans `/var/wetchah/tenants` : ils ne bougent pas. Il faut seulement transférer **la base de la console** (la liste des hôtels) et **la clé** `APP_KEY` vers la nouvelle pile.

1. **Récupérez la clé de l'essai.** Elle est dans le fichier `.env` du dossier de l'essai (par exemple `~/Documents/wetchah/erp_pms/.env`) : `grep APP_KEY .env`. Vous devrez la reprendre telle quelle à l'étape 4.2, à la place d'une clé neuve.
2. **Arrêtez la pile d'essai**, depuis son dossier : `docker compose down`. **Sans l'option -v** : elle effacerait des volumes. Les hôtels ne sont pas arrêtés.

```
cd ~/Documents/wetchah/erp_pms
docker compose down
```

1. Installez la console de production : chapitres 3 et 4, jusqu'à **4.5 inclus**, en réutilisant la clé de l'essai. **Ne faites pas** 4.6 ni 4.7 (la base va être remplacée).
2. Arrêtez la nouvelle console, copiez la base de l'essai dans le volume, puis redémarrez :

```
erp stop
docker run --rm -v wetchah-erp_erp_storage:/donnees -v
~/Documents/wetchah/erp_pms/database:/depart:ro alpine sh -c 'mkdir -p /donnees/database && cp
/depart/database.sqlite /donnees/database/database.sqlite && chown -R 33:33 /donnees/database'
erp start
```

1. Connectez-vous avec **vos anciens identifiants** : vos hôtels sont là. Lancez `erp exec -u www-data app php artisan migrate --force` au cas où la base de l'essai serait en retard.

> **Attention** — Vérifiez que `TENANTS_BASE_PATH` vaut `/var/wetchah/tenants` comme dans l'essai : c'est là que vivent les fichiers des hôtels. Si vos hôtels de l'essai utilisaient un autre dossier, gardez celui-là.

10.2 L'adresse du serveur a changé (nouvelle IP, nom de domaine)

1. Modifiez `APP_URL` et `TENANTS_PUBLIC_HOST` dans le fichier de réglages : `nano /opt/wetchah/erp.env`.
2. Recréez la console : `erp up -d`.
3. **Pour chaque hôtel déjà créé**, faites une mise à jour de version (9.2) : elle réécrit sa configuration avec la nouvelle adresse. Sans cela, l'hôtel garde l'ancienne dans sa configuration (liens, site vitrine).

Les liens « Ouvrir » de la console changent tout de suite ; la configuration interne de chaque hôtel change à sa mise à jour.

10.3 J'ai oublié le mot de passe de mon compte administrateur

```
erp exec -u www-data app php artisan erp:creer-admin --reinitialiser
```

La commande demande l'adresse e-mail du compte, puis un nouveau mot de passe deux fois.

> **Vous devez voir**
> `Mot de passe changé pour marie@exemple.cm.`

Si un autre administrateur existe, il peut aussi le faire depuis la console. Pour un hôtel, c'est le bouton **Réinitialiser le mot de passe** de l'onglet « Comptes administrateurs » (6.5).

10.4 Ajouter d'autres hôtels

Répétez le chapitre 6. Retenez trois règles :

- Chaque hôtel a son **propre port** (8081, 8082, 8083…), dans la plage ouverte au pare-feu.
- Le « nom de base » et le « slug » doivent être **uniques**.
- Chaque hôtel consomme environ 1 Go de mémoire : surveillez `free -h` avant d'en ajouter.

10.5 Mettre la console en HTTPS avec un nom de domaine

Sans cela, tout circule en clair sur des ports (http://IP:8080). Pour un vrai nom (https://erp.exemple.cm), placez un **reverse proxy** devant la console : c'est un petit programme qui reçoit les visiteurs en HTTPS et les transmet à la console. **Nginx Proxy Manager** (installable depuis Portainer) s'occupe aussi des certificats gratuits. Ce guide ne fournit pas ce programme et cette partie n'a pas pu être rejouée ; voici l'essentiel.

1. Faites pointer le nom de domaine vers l'IP du serveur (chez votre registrar de domaine).
2. Dans le fichier de réglages, mettez `APP_URL=https://erp.exemple.cm`, `TRUSTED_PROXIES=*` et `SESSION_SECURE_COOKIE=true`. Puis `erp up -d`.
3. Dans le reverse proxy, créez un hôte qui vise le conteneur `wetchah_erp-app`, port **80**, avec un certificat SSL. Branchez le proxy sur le réseau `pms` pour qu'il trouve la console par son nom.
4. **Autorisez les connexions longues** : la création d'un hôte affiche son journal en continu pendant plusieurs minutes. Désactivez la mise en mémoire des réponses et portez le délai de lecture à au moins 600 secondes.

> **Attention** — **Limite actuelle :** les hôtels restent en HTTP sur leur port (`http://IP:8081`). La console génère leur adresse sans HTTPS ; ils ne passent pas derrière le proxy tant que cette partie n'est pas adaptée.

10.6 Changer de serveur

1. Sur l'ancien serveur : **sauvegardez** (8.1) et copiez les fichiers hors du serveur. Notez la clé `APP_KEY`.
2. Sur le nouveau : chapitres 3 et 4 (jusqu'à 4.5), avec **la même** `APP_KEY`.
3. Restaurez la base de la console (8.4) et le dossier `/var/wetchah/tenants`.
4. Les **données des hôtels** se déplacent avec leurs sauvegardes (menu Sauvegardes de la console). Ce transfert demande de la méthode : faites-le avec une personne qui connaît la plateforme, et testez sur un hôtel d'essai d'abord.

10.7 Tout désinstaller

> **Important** — **Irréversible.** Supprimez d'abord les hôtels depuis la console (fiche → Paramètres → Supprimer l'établissement) : elle retire leurs conteneurs et leurs données. Sauvegardez avant (8.1) si vous hésitez.

```
erp down -v
docker network rm pms
sudo rm -rf /var/wetchah/tenants /opt/wetchah
```

`erp down -v` supprime la console **et son volume** (donc sa base). Sans `-v`, la base est conservée.

11. Problèmes courants

Cherchez le message exact que vous voyez dans la colonne de gauche.

Ce que vous voyez	Cause	Que faire
`required variable APP_KEY is missing a value`	La ligne APP_KEY= du fichier de réglages est vide, ou le raccourci `erp` ne trouve pas le fichier.	Remplir APP_KEY (4.2). Vérifier `alias erp`.
`network pms declared as external, but could not be found`	Le réseau n'existe pas.	`docker network create pms` (3.6).
`port is already allocated` (8080)	Un autre programme utilise ce port — souvent la pile d'essai.	Arrêter l'essai (`docker compose down` dans son dossier) ou changer `ERP_PORT`.
`container name "/wetchah_erp-app" is already in use`	La pile d'essai garde ce nom.	Dans le dossier de l'essai : `docker compose down`.
Page 500 : `Vite manifest not found`	Les écrans ne sont pas compilés. Typique de la pile d'essai.	Chemin B : l'image les contient. Chemin A : annexe A, étape 6.
Journal : `Database file at path … does not exist`	La base n'existe pas. Typique de la pile d'essai.	Chemin B : se corrige tout seul. Chemin A : annexe A, étape 3.
Connexion : `attempt to write a readonly database`	Pile d'essai : le dossier `database/` ou le fichier SQLite appartient à un autre utilisateur que celui de la console (SQLite doit pouvoir écrire dans le **dossier**, pas seulement dans le fichier).	Code à jour : `git pull` puis `docker compose up -d --force-recreate`. Sans mise à jour : `docker compose exec app sh -c 'chmod 777 database && chmod 666 database/database.sqlite'`. Chemin B : ne se produit pas (la base est dans un volume).
Journal : « Préchauffage : /login n'a pas répondu »	La console n'était pas prête au démarrage (base absente, ou lenteur).	Attendre une minute et recharger la page. Si cela persiste, lire le journal (7.1).
`permission denied` sur `docker.sock`	Votre utilisateur n'est pas dans le groupe docker.	3.4, puis se reconnecter.
Création d'un hôte : `mkdir()` : `Permission denied` ou « Impossible de créer le répertoire »	Le dossier des hôtels n'appartient pas à l'utilisateur 33 (Docker le crée au nom de root quand il n'existe pas).	`sudo chown 33:33 /var/wetchah/tenants` (3.7), puis rouvrir la fiche de l'hôte : la création reprend, sans retélécharger les images.
Création d'un hôte : « Impossible de créer le répertoire : `/var/meka-erp/tenants/.compose` »	Pile d'essai ancienne : la console écrit dans un dossier par défaut que le conteneur ne monte pas.	Chemin A : `git pull` et `docker compose up -d` (le dossier par défaut devient `/var/wetchah/tenants`). Sans mise à jour : `docker compose exec app sh -c 'echo TENANTS_BASE_PATH=/var/wetchah/tenants >> .env'`, `docker compose restart app`, puis rouvrir la fiche de l'hôtel.

Ce que vous voyez	Cause	Que faire
Création d'un hôte avec le module GRC : « Image GRC distante non disponible »	Votre console utilise une ancienne adresse d'image (`ghcr.io/wetchah/wetchah_grc`, refusée par GHCR).	Mettre à jour le code (`git pull`, 9.1). L'adresse correcte est `ghcr.io/clyde237/wetchah_grc` ; en attendant, ajouter `REGISTRY_IMAGE_GRC=ghcr.io/clyde237/wetchah_grc` au fichier de réglages.
Les liens « Ouvrir » mènent à `localhost`	`TENANTS_PUBLIC_HOST` n'est pas renseigné.	Le définir (4.2), `erp up -d`, puis mettre à jour chaque hôtel (10.2).
Création d'un hôte : « Port … déjà utilisé »	Le port choisi est pris.	Choisir un autre port dans la plage (`docker ps` montre les ports pris).
La création reste sur « Récupération de l'image »	Connexion lente vers `ghcr.io`.	Patienter : la reprise se fait par morceaux. Sinon revoir 3.8.
La création s'est arrêtée : j'avais fermé la page	Le suivi en direct a été interrompu.	Rouvrir la fiche de l'hôtel : la création reprend (6.4).
Un hôte est injoignable depuis un autre poste	Port fermé dans le pare-feu, ou hors de la plage ouverte.	3.5. Tester avec `curl http://IP_DU_SERVEUR:PORT/login` depuis le poste.
Le site vitrine d'un hôte refuse ses formulaires	Son adresse d'origine pointe vers l'ancienne adresse.	Renseigner `TENANTS_PUBLIC_HOST`, puis mettre à jour l'hôte (9.2).
Bouton « Mettre à jour le code » : « Le code de cette console est dans l'image Docker »	Console de production : le code est dans l'image, pas dans un dépôt git.	Mettre à jour en reconstruisant l'image (9.1).
Après `git pull`, rien ne change	L'image n'a pas été reconstruite, ou le numéro de version n'a pas été changé.	9.1, étapes 2 à 4.
Connexion à la console refusée	Mauvais mot de passe, ou le champ « Identifiant » doit contenir l'e-mail.	10.3 pour réinitialiser.
Le disque se remplit	Anciennes images et journaux.	`docker system df`, puis `docker image prune` **sans** l'option -a (les images des hôtels doivent rester).
Après un redémarrage du serveur, rien ne tourne	Docker ne démarre pas tout seul.	`sudo systemctl enable --now docker`.

11.1 Quatre commandes pour comprendre

Avant de demander de l'aide, lancez ces commandes et notez ce qu'elles répondent :

```
erp ps
erp logs --tail 40 app
erp exec app tail -n 40 storage/logs/laravel.log
docker inspect wetchah_erp-app --format '{{json .State.Health}}'
```

12. Questions fréquentes

Question	Réponse
La console tourne-t-elle sur SQLite ou PostgreSQL ?	**SQLite.** Chaque hôtel, lui, a son PostgreSQL. Voir 1.2. Pour vérifier : `erp exec app printenv DB_CONNECTION` répond `sqlite`.
Faut-il installer PostgreSQL sur le serveur ?	Non. Les bases des hôtels sont créées par la console, dans leurs conteneurs.
Pourquoi la console a-t-elle besoin du socket Docker ?	C'est par là qu'elle crée, arrête et met à jour les hôtels. C'est aussi pourquoi le serveur doit lui être réservé.
Peut-on gérer les hôtels depuis Portainer ?	Les **regarder**, oui. Les créer, arrêter, supprimer ou mettre à jour : non, passez par la console (6.6).
Combien d'hôtels sur un serveur ?	Environ 1 Go de mémoire par hôtel. Un serveur de 8 Go en tient raisonnablement 5 à 6.
Les hôtels ont-ils besoin d'Internet ?	Pour être créés ou mis à jour, oui (téléchargement des images). Pour travailler au quotidien, ils fonctionnent sur le réseau local.
Où sont les données ?	Console : volume `wetchah-erp_erp_storage`. Hôtels : volumes `meka-erp-NOM_meka_erp_NOM_pgdata` et `meka-erp-NOM_meka_erp_NOM_storage`. Fichiers des hôtels : `/var/wetchah/tenants`.
Où trouver la version installée ?	`erp ps` affiche le numéro de l'image. La fiche d'un hôte, menu Paramètres, indique la version de l'hôtel.
Puis-je installer sur un autre système que Ubuntu ?	Les commandes de Docker sont les mêmes sur d'autres Linux ; seules la préparation du serveur (chapitre 3) et le nom des outils changent. Ce guide n'a été vérifié que sur Ubuntu.
Où est le journal des installations de la console ?	`erp logs app` pour le démarrage, et `storage/logs/laravel.log` pour les erreurs de l'application.

13. Liste de contrôle

Serveur

- ☐ Ubuntu à jour, heure réglée sur Africa/Douala (3.1 à 3.3).
- ☐ `docker ps` fonctionne sans `sudo` (3.4).
- ☐ Pare-feu : 8080, 8081–8199 et 9081–9199 ouverts (3.5).
- ☐ Réseau `pms` créé (3.6).
- ☐ `/var/wetchah/tenants` créé, propriétaire 33:33 (3.7).
- ☐ `docker pull ghcr.io/adrien-stage/villa_b:latest` réussit (3.8).

Console

- ☐ Code téléchargé dans `/opt/wetchah/wetchah_erp` (4.1).
- ☐ Fichier de réglages créé, droits 600, `APP_KEY` et `TENANTS_PUBLIC_HOST` renseignés (4.2).
- ☐ `APP_KEY` et clés VAPID copiées **hors du serveur** (4.2, 4.3).
- ☐ `erp ps` affiche `healthy` (4.5).
- ☐ `migrate --force` lancé, **sans** `db:seed` (4.6).
- ☐ Compte administrateur créé, connexion réussie (4.7, 4.8).
- ☐ Socket Docker en `srw-rw----` (4.9).

Premier hôtel

- ☐ Hôtel créé, état « Conteneurs actifs » (6.4).
- ☐ Lien d'accès affiché avec l'adresse du serveur, pas localhost (6.4).
- ☐ Mot de passe PostgreSQL remplacé à l'écran 2 (6.2).
- ☐ Compte administrateur de l'hôtel créé et transmis (6.5).
- ☐ L'hôtel s'ouvre depuis un autre poste du réseau.

Exploitation

- ☐ Sauvegarde nocturne planifiée (8.2).
- ☐ Copie hors serveur en place (8.3).
- ☐ Un essai de restauration réalisé (8.3).
- ☐ Redémarrage du serveur testé : la console et les hôtels remontent seuls (7.2).

Annexe A — Le chemin A : essai rapide

Ces étapes lancent la console avec le fichier `docker-compose.yml` (pile d'essai) depuis un dossier cloné. Elles conviennent pour essayer ou montrer la plateforme. **Ne les utilisez pas pour un serveur de production** : la pile d'essai tourne en mode développement et ouvre le socket Docker à tous les utilisateurs de la machine.

Placez-vous d'abord dans le dossier du code (celui où se trouve `docker-compose.yml`), par exemple `cd ~/Documents/wetchah/erp_pms`. Dans cette pile, le dossier est directement lu par le conteneur : les fichiers qui manquent après un clonage doivent être créés à la main.

1. **Démarrer.** Créez le réseau (3.6) puis lancez la pile :

```
docker network create pms
docker compose up -d --build
```

1. **Créer le fichier de configuration** (sans écraser un fichier existant) :

```
docker compose exec app sh -c 'cp -n .env.example .env'
```

1. **Créer le fichier de la base SQLite** et lui donner les bons droits. C'est ce qui manque quand le journal affiche « Database file at path … does not exist » :

```
docker compose exec app sh -c 'touch database/database.sqlite && chown -R www-data:www-data
database'
```

1. **Générer la clé de l'application** :

```
docker compose exec app php artisan key:generate
```

1. **Créer les tables** :

```
docker compose exec app php artisan migrate --force
```

1. **Compiler les écrans.** C'est ce qui manque quand la page de connexion répond « Vite manifest not found » :

```
docker compose exec app npm run build
docker compose exec app chown -R www-data:www-data public/build
```

1. **Créer votre compte administrateur** (nécessite le code à jour : `git pull` d'abord) :

```
docker compose exec -u www-data app php artisan erp:creer-admin
```

Ouvrez ensuite `http://IP_DU_SERVEUR:8080`. Pour passer ensuite en production, lisez le chapitre 10.1.

> **À savoir** — Pour un compte d'essai très rapide, `php artisan db:seed` crée `admin` / `admin`. **Uniquement sur une machine d'essai, jamais exposée.**

Annexe B — Les réglages du fichier

Le modèle complet est `.env.production.example`. Voici toutes les variables, dans l'ordre du fichier.

Variable	Rôle	Valeur
`APP_KEY`	Clé secrète de la console. **Obligatoire.**	`base64:…`
`APP_URL`	Adresse de la console	http://IP:8080
`ERP_PORT`	Port de la console sur le serveur	8080
`TENANTS_PUBLIC_HOST`	IP ou domaine sous lequel on atteint les hôtels	IP du serveur
`TENANTS_BASE_PATH`	Dossier des fichiers des hôtels (même chemin dans le conteneur)	/var/wetchah/tenants
`TZ`	Fuseau horaire	Africa/Douala
`TRUSTED_PROXIES`	Adresses du reverse proxy (* si c'est le seul à joindre la console). Vide sans proxy.	(vide)
`SESSION_SECURE_COOKIE`	`true` uniquement en HTTPS	false
`VAPID_SUBJECT, VAPID_PUBLIC_KEY, VAPID_PRIVATE_KEY`	Clés des notifications (4.3)	—
`MAIL_MAILER, RESEND_API_KEY, MAIL_FROM_ADDRESS`	Envoi de courriels. `log` = rien n'est envoyé, tout est journalisé.	log
`ERP_VERSION`	Numéro de version de l'image (9.1). Absent = `latest`.	(absent)
`REPORTING_SECRET, ASSISTANCE_SECRET`	Anciens secrets communs, inutiles sur une installation neuve.	(vide)
`REGISTRY_IMAGE, REGISTRY_IMAGE_WEB, REGISTRY_IMAGE_GRC`	Images de l'hôtel, de son site vitrine et du module de contrôle de gestion (par défaut les images publiques de GHCR).	(défaut)

À savoir — Dans le fichier, une valeur qui contient un signe `$` doit l'écrire `$$`. La clé `APP_KEY` n'en contient jamais.

Annexe C — Ce qui a été vérifié

Ce guide a été écrit en exécutant les commandes pour de vrai, sur un serveur Docker Linux, dans un environnement isolé de vos hôtels. Voici ce qui a été rejoué, et ce qui ne l'a pas été.

Étape du guide	Vérifié	Comment
Fabrication de l'image, démarrage, état « healthy » (4.4, 4.5)	Oui	Avec un fichier de réglages et le raccourci équivalent à `erp`
Clé fabriquée par `openssl` (4.2)	Oui	Acceptée par la console, connexion réussie
Migration et création du compte (4.6, 4.7)	Oui	Dont la saisie interactive dans un vrai terminal
Connexion à la console, Supervision (4.8)	Oui	Captures de ce guide
Socket Docker en `root` : `docker` et `docker ps` sous www-data (4.9)	Oui	
Création d'un hôtel de bout en bout (6)	Oui	Image téléchargée, conteneurs démarrés, hôtel en ligne, connexion à l'hôtel
Fermeture de la page pendant la création (6.4)	Oui	Rouverte : la création reprend et se termine
Création du compte administrateur d'un hôtel (6.5)	Oui	Et connexion à l'hôtel avec ce compte
Sauvegarde, rotation des copies (8.1)	Oui	
Restauration de la base de la console (8.4, 10.1)	Oui	Fichier copié dans le volume, console redémarrée, données retrouvées
Mise à jour numérotée et retour arrière (9.1)	Oui	Les données survivent au changement d'image
Changement de l'adresse d'un hôtel (10.2)	Oui	Après la mise à jour de version, le fichier de l'hôtel porte la nouvelle adresse
Réinitialisation du mot de passe (10.3)	Oui	L'ancien mot de passe est refusé, le nouveau accepté
Écrans de Portainer (chapitre 5)	Non	Libellés à vérifier sur votre version de Portainer
HTTPS et reverse proxy (10.5)	Non	Variables vérifiées séparément : les adresses des ressources passent en https derrière un proxy
Déplacement des données d'un hôtel vers un autre serveur (10.6)	Non	À faire avec précaution et un hôtel d'essai
Planification par `cron` (8.2)	Non	Le programme est vérifié ; l'enregistrement dans `crontab` ne l'est pas